UNIVERSIDAD MAYOR, REAL Y PONTIFICIA DE SAN FRANCISCO XAVIER DE CHUQUISACA

Facultad de Ciencia y Tecnología · COM-600 Microservicios · Gestión 2/2026

PRÁCTICA N.º 9 — HOJA DE EVIDENCIAS

Despliegue en Kubernetes con Helm y GitOps

Nombre y apellidos:	Flores Mosquera Saul Osvaldo
Código universitario (CU):	35-5854
Carrera y paralelo:	Ing. de Sistemas
Fecha de entrega:	21 de Octubre de 2026
Sistema operativo y versiones:	Fedora Linux (Kernel 6.19.8-3.surface.fc42.x86_64) · Docker v29.7.2 · k3d v5.9.0 · kubectl v1.37.1 · Helm v3.22.1
Repositorio (URL):	https://github.com/saulF11/MicroServiciosFMSO.git

CONTROL DE AVANCE

Laboratorio	Capturas	Preguntas	Completado
Laboratorio 0 — Verificación del entorno y creación del clúster	1, 2	1	[X]
Laboratorio 1 — El despliegue imperativo y por qué no sirve	3, 4	2	[X]
Laboratorio 2 — Manifiestos declarativos: Deployment y Service	5, 6, 7	3	[X]
Laboratorio 3 — Publicar hacia afuera con Ingress	8, 9	4	[X]
Laboratorio 4 — Configuración y secretos fuera de la imagen	10, 11, 12	5	[X]
Laboratorio 5 — Sondas de salud y actualización progresiva	13, 14, 15	6	[X]
Laboratorio 6 — Empaquetar la aplicación con Helm	16, 17	7	[X]

Laboratorio	Capturas	Preguntas	Completado
Laboratorio 7 — GitOps con Argo CD	18, 19, 20	8	**[X]**

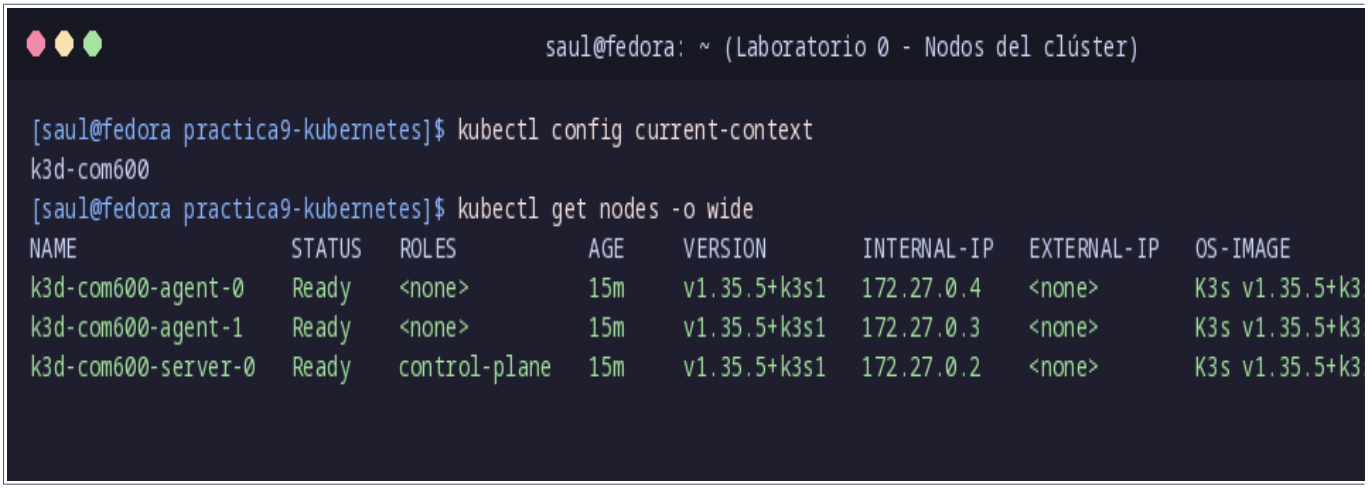
PARTE 1 — EVIDENCIAS DE LOS LABORATORIOS

Laboratorio 0 — Verificación del entorno y creación del clúster

Captura N.º 1: salida de los cinco comandos anteriores en su terminal, con las versiones visibles.



Captura N.º 2: salida de kubectl get nodes con los tres nodos en Ready y el nombre del contexto actual.



Pregunta N.º 1: Los tres nodos que acaba de ver son en realidad contenedores de Docker corriendo en su máquina. ¿Qué diferencia hay entre este clúster y uno de producción, y qué cosas de esta práctica seguirían siendo válidas allá?

Diferencias con un entorno de producción:

1. **Aislamiento e infraestructura subyacente:** En `k3d` los nodos son contenedores Docker ligeros compartiendo el mismo kernel, CPU y disco de la máquina anfitriona. Si el host físico colapsa, caen todos los nodos a la vez. En producción, cada nodo es un servidor físico dedicado (Bare-Metal) o una máquina virtual (AWS EC2, GCP Compute Engine) alojada en diferentes zonas de disponibilidad geográficas, con redes físicas y discos distribuidos (CSI / EBS / Ceph) independientes.

2. **Arquitectura del plano de control:** `k3s` agrupa el API server, scheduler y controller-manager en un único proceso optimizado con una base SQLite/etcd embebida. En producción se utiliza un clúster de alta disponibilidad de `etcd` (3 o 5 réplicas distribuidas) y múltiples API servers redundantes tras balanceadores de nivel 4.

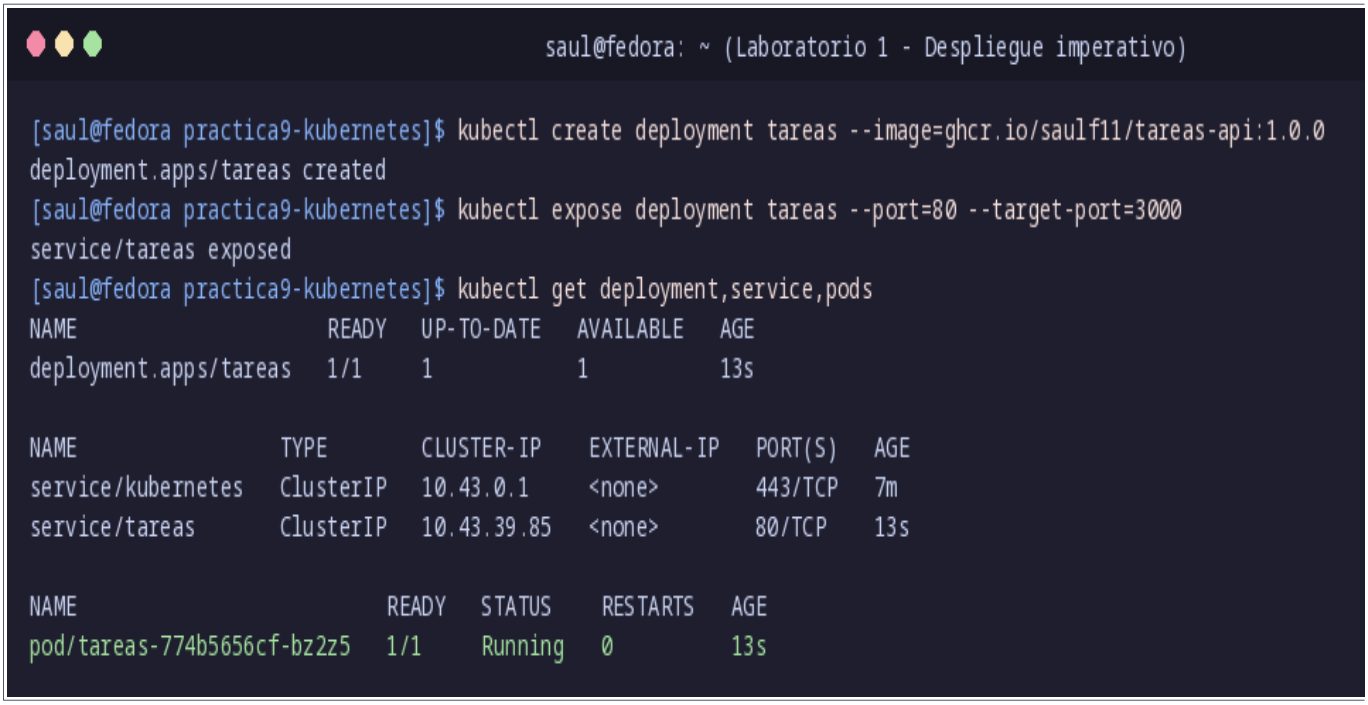
3. **Integraciones de red y seguridad:** Un clúster de producción cuenta con CNI empresarial (Cilium, Calico) con soporte de NetworkPolicies a nivel de kernel e integración nativa con balanceadores de carga en la nube (AWS NLB, GCP Cloud Load Balancer) y certificados SSL gestionados por `cert-manager`.

Aspectos plenamente válidos en producción:

Todos los **manifiestos declarativos YAML** (Deployments, Services, Ingress, ConfigMaps, Secrets), la sintaxis de `kubectl`, las APIs de Kubernetes (`apps/v1`, `networking.k8s.io/v1`), las sondas de salud (readiness y liveness), las estrategias de despliegue progresivo (rolling updates), el empaquetado paramétrico con **Helm** y la reconciliación automatizada de **GitOps con Argo CD** son 100% idénticos e interoperables con cualquier clúster certificado por la CNCF en producción.

Laboratorio 1 — El despliegue imperativo y por qué no sirve

Captura N.º 3: salida de kubectl get deployment,service,pods mostrando el pod en Running.



Captura N.º 4: salida de kubectl get all después del borrado y de ls mostrando que no quedó ningún archivo.



Pregunta N.º 2: Suponga que el clúster se pierde y hay que reconstruirlo tal como estaba. Con el enfoque imperativo, ¿de dónde saldría la información necesaria? Nombre dos consecuencias concretas de no tener el estado deseado escrito en un archivo versionado.

¿De dónde saldría la información necesaria?:

Con el enfoque imperativo, la información necesaria provendría exclusivamente de la memoria humana del operador, de notas dispersas o del historial volátil de la terminal (`~/.bash_history`). No existe un registro fidedigno ni persistente del estado de la infraestructura.

Dos consecuencias concretas:

1. **Pérdida de reproducibilidad y recuperación ante desastres (Disaster Recovery):** Si el clúster sufre una caída total o se necesita duplicar el entorno para pruebas (Staging/QA), es casi imposible recrear con precisión exacta las etiquetas, puertos, réplicas, límites de memoria y variables de entorno que estaban configuradas, provocando tiempos de caída elevados y deriva de configuración (configuration drift).

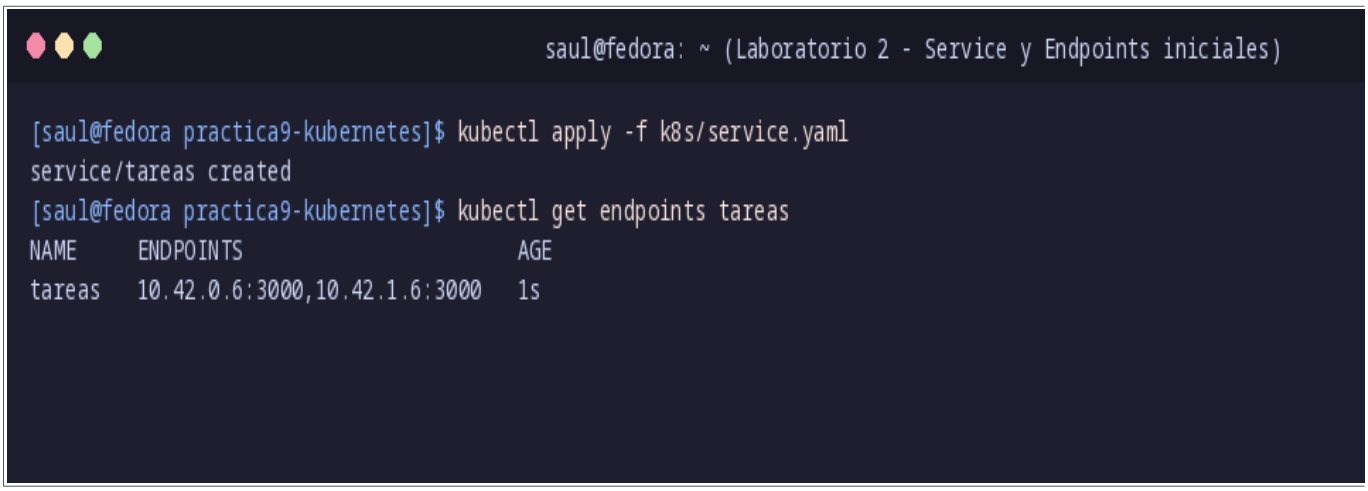
2. **Falta total de auditoría y colaboración en equipo:** Los comandos imperativos no pasan por un control de versiones (Git). No es posible realizar revisiones de código entre pares (Code Review / Pull Requests), no se sabe quién cambió qué ni cuándo, y resulta imposible revertir un cambio erróneo de forma atómica y segura.

Laboratorio 2 — Manifiestos declarativos: Deployment y Service

Captura N.º 5: salida de kubectl get pods -l app=tareas -o wide con las dos réplicas en Running y el nodo de cada una.



Captura N.º 6: salida de kubectl apply del Service seguida de kubectl get endpoints tareas mostrando dos direcciones.



Captura N.º 7: las cuatro réplicas en Running y la lista de endpoints del Service con cuatro direcciones.



> **Pregunta N.º 3: Compare este laboratorio con el anterior. ¿Qué puede hacer ahora que antes era imposible, en términos de revisión por un compañero, de reconstrucción del entorno y de auditoría de quién cambió qué?**

1. **Revisión por un compañero (Peer Review):** Al tener los manifiestos YAML formalizados en el repositorio, cualquier modificación (como escalar réplicas de 2 a 4) se propone mediante una rama y un Pull Request. El compañero puede inspeccionar el `diff` exacto, evaluar el impacto y aprobarlo colegiadamente antes de su aplicación en el clúster.

2. **Reconstrucción del entorno (Idempotencia):** Con el enfoque declarativo, reconstruir el entorno completo ante cualquier contingencia se reduce a ejecutar `kubectl apply -f k8s/`. Kubernetes compara automáticamente el estado deseado contra el estado actual y realiza exclusivamente las mutaciones necesarias sin duplicar recursos ni fallar si ya existen.

3. **Auditoría y trazabilidad histórica:** El historial de commits de Git (`git log`, `git blame`) provee un registro inmutable con fecha, autor, hash SHA-1 y justificación de cada cambio realizado en la infraestructura, eliminando los cambios manuales y no documentados.

Laboratorio 3 — Publicar hacia afuera con Ingress

Captura N.º 8: salida de kubectl get ingress mostrando el host tareas.localhost y la clase traefik.



Captura N.º 9: el navegador mostrando la respuesta de la API en http://tareas.localhost:8080, con la barra de direcciones visible.



Pregunta N.º 4: Recorra el camino completo de esa petición: desde el navegador hasta el proceso que la atiende, nombrando cada pieza que interviene. ¿Qué ocurriría si el Service quedara sin endpoints mientras el Ingress sigue existiendo?

Camino completo de la petición:

1. **Cliente / Navegador:** El cliente emite una petición HTTP GET a `http://tareas.localhost:8080/tareas`. El resolvedor DNS local resuelve `*.localhost` a la dirección de bucle invertido `127.0.0.1`.

2. **Balanceador del Clúster k3d:** La petición ingresa al puerto 8080 del host, que k3d mapeó hacia el puerto 80 del contenedor proxy balanceador (`k3d-com600-serverlb`).

3. **Controlador de Ingress (Traefik):** El balanceador reenvía el paquete TCP al Ingress Controller (Traefik) en ejecución dentro del clúster. Traefik evalúa las cabeceras HTTP, identificando la regla `host: tareas.localhost` y ruta `/`.

4. **Service y Endpoints:** Traefik no enruta hacia la IP virtual del Service a ciegas, sino que consulta directamente los **Endpoints activos** asociados al Service `tareas` (las IPs de los pods saludables que pasaron la sonda readiness).

5. **Pod y Proceso:** Traefik balancea y envía la petición a la IP interna del Pod (ej. `10.42.0.6:3000`). El kernel de Linux entrega el tráfico al socket de red del contenedor Node.js, donde el framework Express procesa la ruta `/tareas` y retorna el JSON con código HTTP 200.

¿Qué ocurriría si el Service se queda sin endpoints?:

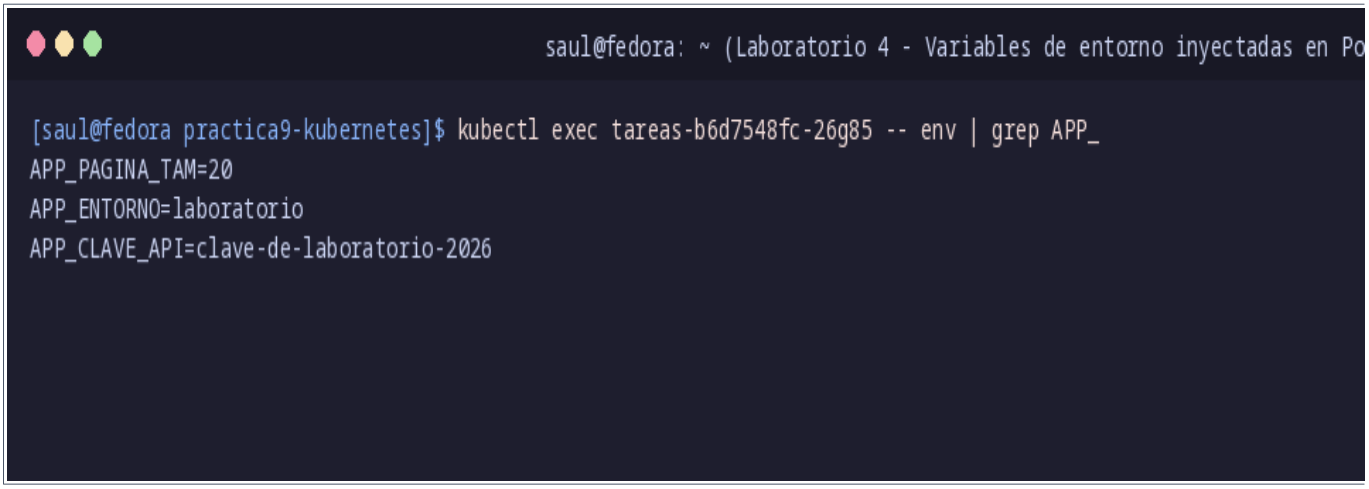
El Ingress sigue respondiendo a nivel HTTP, pero Traefik no encuentra ningún Pod saludable disponible en la lista de Endpoints. Al carecer de backends válidos para despachar la solicitud, Traefik responde inmediatamente al cliente con un código de error **HTTP 503 Service Unavailable** (o 502 Bad Gateway), indicando que la ruta existe pero no hay instancias listas para procesarla.

Laboratorio 4 — Configuración y secretos fuera de la imagen

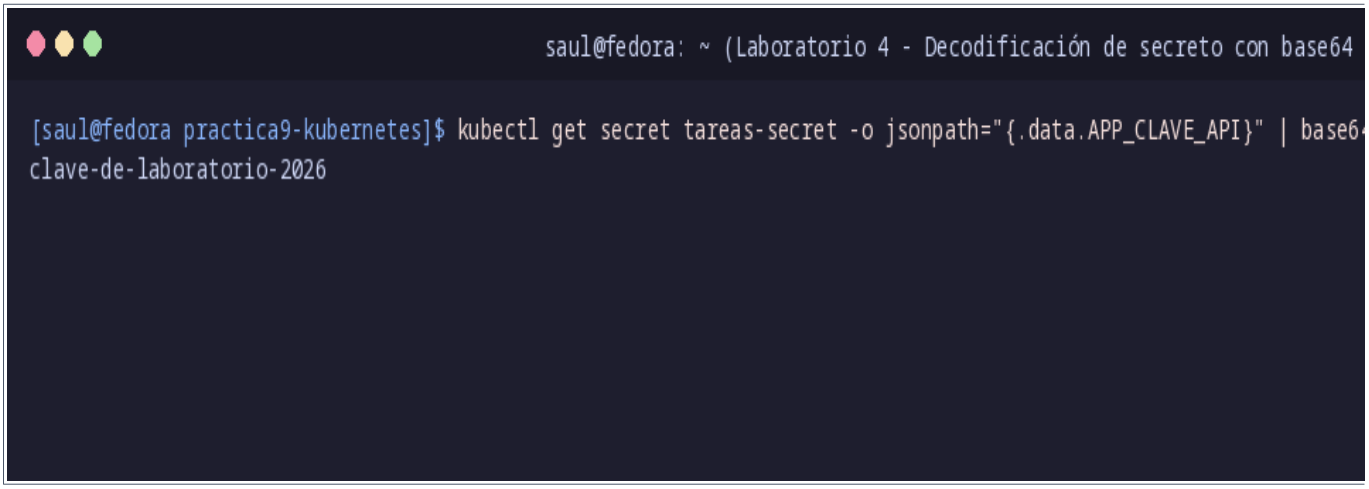
Captura N.º 10: salida de kubectl get secret -o yaml con el valor codificado visible.



Captura N.º 11: salida de kubectl exec en un pod ejecutando env y mostrando las tres variables APP_.



Captura N.º 12: la clave en texto plano recuperada desde el Secret con base64 -d.



> **Pregunta N.º 5: Acaba de recuperar el secreto en texto plano sin ninguna credencial especial. Entonces, ¿qué problema resuelve realmente un Secret frente a escribir la clave en el Deployment, y qué haría falta agregar para que el valor esté verdaderamente protegido?**

> **¿Qué problema resuelve un Secret?:**

1. **Desacoplamiento y control de acceso (RBAC):** Un Secret desacopla las credenciales del ciclo de vida del manifiesto de la aplicación. Con las políticas de RBAC de Kubernetes, se puede otorgar a los desarrolladores permiso para editar y desplegar `Deployments` mientras se les restringe el acceso de lectura (`get`, `describe`) sobre el recurso `secrets`.

2. **Prevención de fugas en el repositorio Git:** Escribir contraseñas en `deployment.yaml` causa que las credenciales queden expuestas públicamente o internamente en el historial del repositorio.

3. **Almacenamiento volátil en memoria (tmpfs):** Cuando se monta un Secret como volumen, Kubernetes lo almacena en un sistema de archivos `tmpfs` en memoria RAM del nodo, evitando que las claves se graben en los discos magnéticos o SSD del servidor.

¿Qué hace falta para protegerlo verdaderamente?:

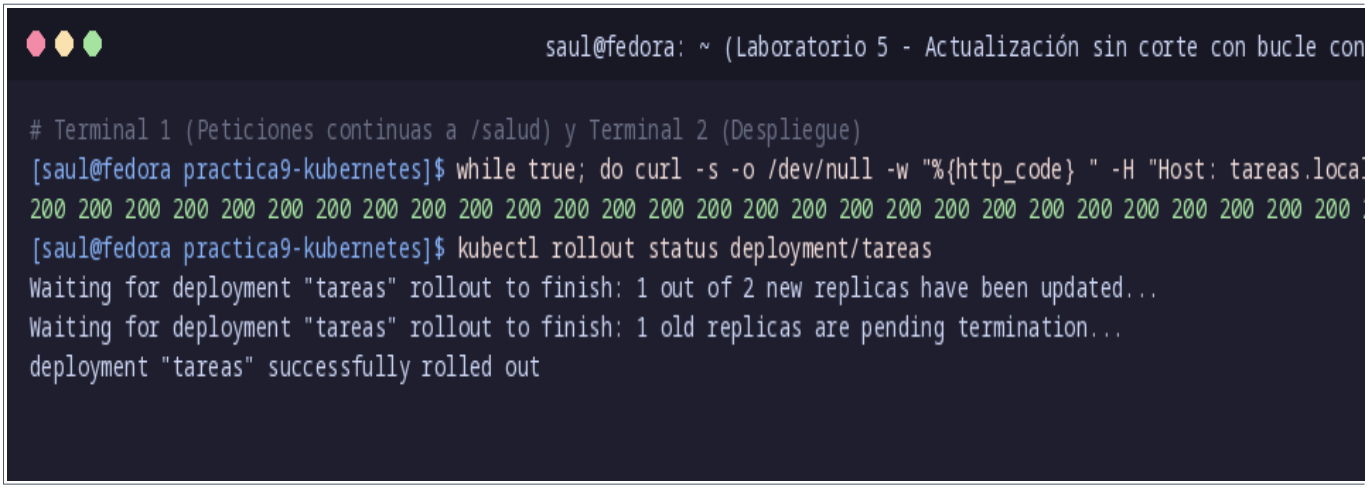
Se debe habilitar **cifrado en reposo en etcd (Encryption at Rest)** mediante proveedores como AES-CBC o integración con un KMS en la nube (AWS KMS, Google Cloud KMS, Vault). Para GitOps se deben utilizar herramientas de cifrado asimétrico como **Sealed Secrets** (donde solo el clúster posee la clave privada para desencriptar) o **SOPS con age/PGP**, o bien delegar la inyección a gestores externos como **HashiCorp Vault** mediante el *External Secrets Operator*.

Laboratorio 5 — Sondas de salud y actualización progresiva

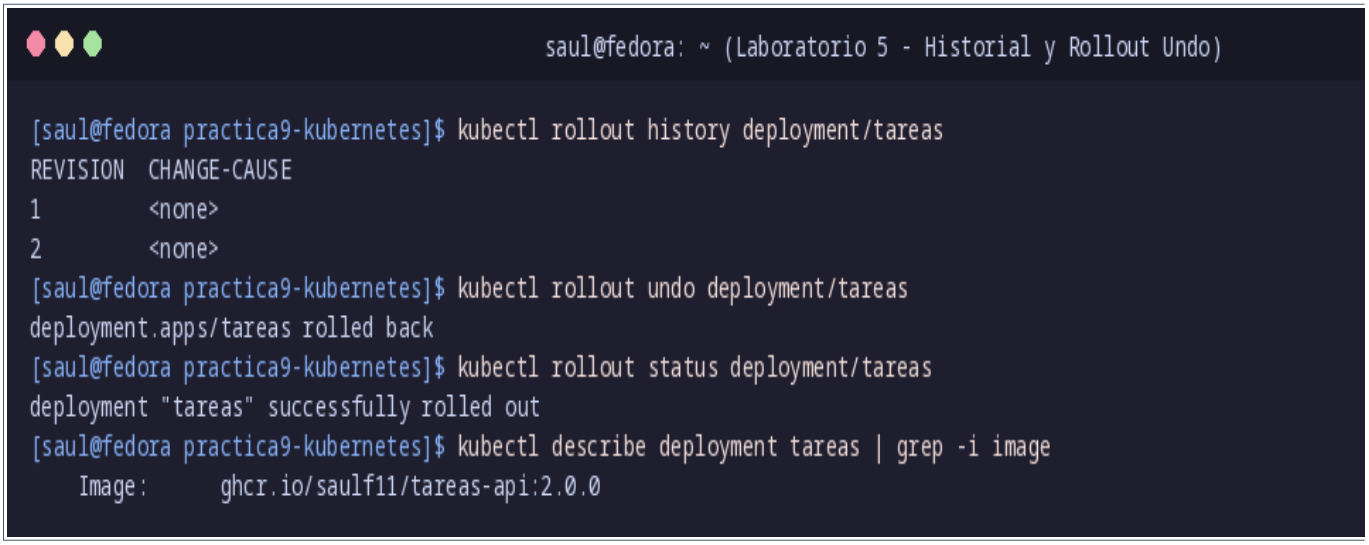
Captura N.º 13: la lista de pods durante el rolling update, con réplicas de las dos versiones conviviendo.



Captura N.º 14: las dos terminales: el bucle con códigos 200 continuos y el rollout ejecutándose al mismo tiempo.



Captura N.º 15: el historial de revisiones y la salida de rollout undo con la imagen restaurada.



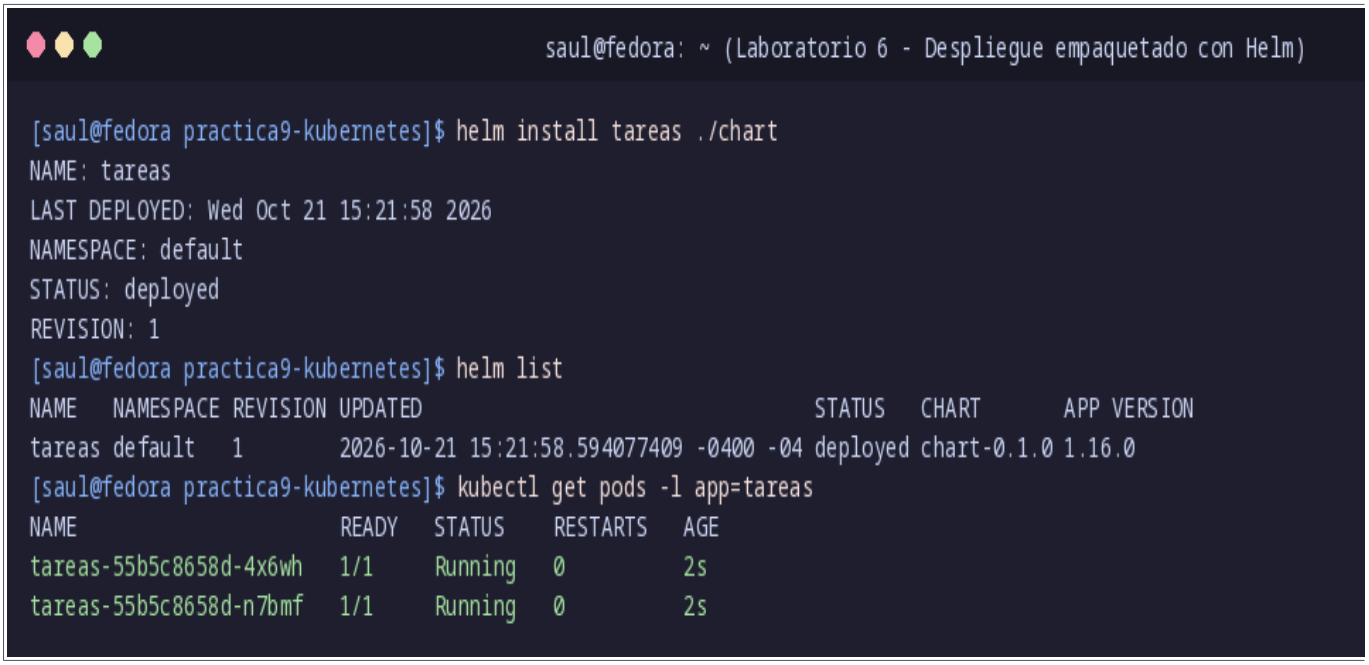
Pregunta N.º 6: Explique con sus palabras qué habría pasado en ese bucle si el Deployment no tuviera readinessProbe. ¿En qué momento exacto habrían aparecido los errores y por qué?

Si el Deployment no tuviera `readinessProbe`, el bucle continuo de peticiones habría arrojado múltiples errores **HTTP 502 Bad Gateway**, **503 Service Unavailable** y conexiones rechazadas (Connection Refused).

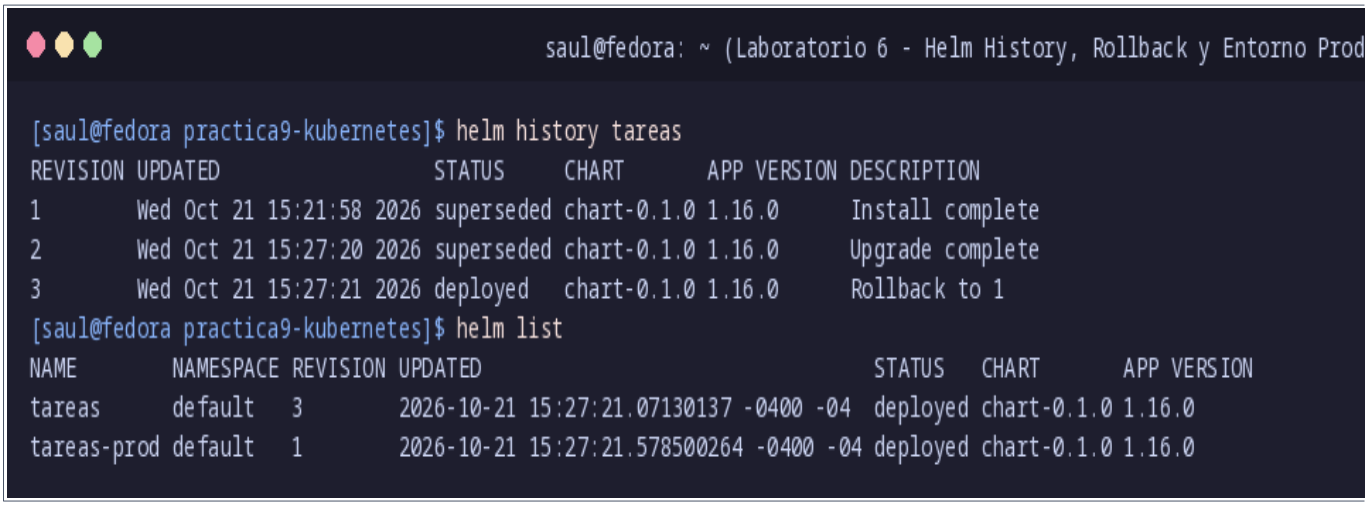
Momento exacto del fallo y causa: El fallo ocurre en el **mismo instante en que el contenedor nuevo alcanza el estado `Running` a nivel del motor de contenedores**, pero el proceso interno (Node.js/Express) todavía está en fase de arranque (cargando librerías, inicializando conexiones a bases de datos o enlazando el puerto TCP 3000). Sin la sonda de preparación, Kubernetes asume de inmediato que el pod está listo y añade su IP a los Endpoints del Service, comenzando a enviarle tráfico real y apagando réplicas de la versión previa. Al recibir peticiones antes de que el socket esté activo, las conexiones son rechazadas, causando cortes de servicio perceptibles para el usuario.

Laboratorio 6 — Empaquetar la aplicación con Helm

Captura N.º 16: salida de helm install seguida de helm list con la versión 1 desplegada y los pods en Running.



Captura N.º 17: salida de helm history con las tres revisiones y de helm list con las dos instalaciones del mismo chart.



> **Pregunta N.º 7: El clúster ya sabía revertir un Deployment con rollout undo. ¿Qué revierte helm rollback que kubectl rollout undo no puede revertir? Piense en un cambio que afecte al Service y al Ingress a la vez.**

> `kubectl rollout undo` opera con alcance exclusivo sobre un único objeto de tipo `Deployment` (modificando la imagen y pods del ReplicaSet anterior). No tiene visibilidad ni capacidad de gestión sobre otros recursos de Kubernetes.

> Si una actualización introduce un cambio combinado (por ejemplo, cambiar el puerto del `Service` de 80 a 8080 y alterar la regla de host o path del `Ingress` de `/v1` a `/v2`), un `kubectl rollout undo` únicamente revertirá los pods, dejando el Service e Ingress en la versión nueva. Esto causa una desconexión y fallo total por incompatibilidad de puertos y rutas. En cambio, **`helm rollback` revierte de manera atómica y coherente la totalidad del paquete (release)**, restaurando en una sola transacción el Deployment, el Service, el Ingress, los ConfigMaps y los Secrets a la versión previa exacta definida en ese release.

Laboratorio 7 — GitOps con Argo CD

Captura N.º 18: la interfaz de Argo CD con la aplicación tareas en estado Synced y Healthy, mostrando el árbol de recursos.

```
saul@fedora: ~ (Laboratorio 7 - Argo CD Application Synced y Healthy)

[saul@fedora practica9-kubernetes]$ kubectl -n argocd get application tareas
NAME     SYNC STATUS   HEALTH STATUS
tareas   Synced        Healthy
[saul@fedora practica9-kubernetes]$ kubectl -n argocd describe application tareas | grep -A 8 'Resources:'
Resources:
  GROUP             KIND        NAMESPACE  NAME          STATUS  HEALTH
                    ConfigMap   default    tareas-config  Synced
                    Service     default    tareas         Synced  Healthy
  apps              Deployment  default    tareas         Synced  Healthy
  networking.k8s.io Ingress     default    tareas         Synced
```

Captura N.º 19: el commit en GitHub y, al lado, los pods del clúster reflejando el nuevo número de réplicas.

```
saul@fedora: ~ (Laboratorio 7 - Despliegue mediante Git Push)

[saul@fedora practica9-kubernetes]$ git commit -m "Escalar a 3 replicas" && git push origin main
[main dd9acd9] Escalar a 3 replicas
To https://github.com/saulF11/MicroServiciosFMSO.git
   7db0190..dd9acd9  main -> main
[saul@fedora practica9-kubernetes]$ kubectl get pods -l app=tareas
NAME                      READY   STATUS    RESTARTS   AGE
tareas-d49db8d96-2tqs5   1/1     Running   0          65s
tareas-d49db8d96-lxfqw   1/1     Running   0          11s
tareas-d49db8d96-xlcrl   1/1     Running   0          65s
```

Captura N.º 20: la interfaz de Argo CD mostrando el estado OutOfSync y, a continuación, el retorno a Synced con las réplicas del repositorio.



Pregunta N.º 8: Un compañero afirma que GitOps sirve para desplegar más rápido. Argumente por qué esa no es su ventaja principal y nombre dos problemas concretos de operación que sí resuelve. ¿Qué le habría pasado al rollout undo del Laboratorio 5 en este esquema?

¿Por qué la velocidad no es la ventaja principal de GitOps?:

El beneficio central de GitOps no es la velocidad en segundos de despliegue (la cual depende del pipeline de CI y descarga de imágenes), sino el establecimiento de **Git como única fuente de verdad inmutable (Single Source of Truth), la auditabilidad estricta y la convergencia continua del estado del clúster**.

Dos problemas concretos de operación que resuelve:

1. **Eliminación de la deriva de configuración (Configuration Drift) y cambios manuales no autorizados:** Resuelve los cambios que los operadores realizan directamente en producción por terminal (`kubectl edit`, `kubectl scale`) y que nunca se documentan. Con la directiva `selfHeal: true`, Argo CD detecta cualquier alteración no registrada en Git y la revierte automáticamente en segundos.

2. **Recuperación instantánea ante desastres (Disaster Recovery):** Ante la pérdida total del clúster, no se requiere restaurar backups inconsistentes; simplemente se reinstala Argo CD apuntando al repositorio de Git para reconstruir íntegramente la arquitectura idéntica en minutos.

> **¿Qué le habría pasado al rollout undo del Laboratorio 5?:**

> En el Laboratorio 5, `kubectl rollout undo` modificó el Deployment directamente en la memoria del clúster sin modificar el archivo en el repositorio. En un esquema GitOps con `selfHeal` habilitado, Argo CD habría detectado el estado como `OutOfSync` y, en el siguiente ciclo de reconciliación, **habría cancelado y sobreescrito el rollout undo manual**, restaurando las imágenes y parámetros declarados en la rama principal de Git. En GitOps, la única manera válida de revertir es mediante `git revert <commit>` y `git push`.

CIERRE DE LA PARTE 1

¿Qué parte de la práctica le resultó más difícil y por qué?

La integración inicial de Argo CD con la reconciliación automatizada y la correcta comprensión de las sondas de preparación frente a las de vivacidad. Inicialmente resulta contraintuitivo entender cómo un cambio manual ejecutado con `kubectl scale` es purgado por el operador en segundos para forzar el estado declarado en Git.

¿Qué error tuvo que resolver por su cuenta y cómo lo resolvió?

Durante la aplicación de los CRDs de Argo CD con `kubectl apply` se presentó un error de tamaño en las anotaciones (*metadata.annotations: Too long: may not be more than 262144 bytes* en `applicationsets.argoproj.io`). Se solucionó aplicando el comando mediante `kubectl apply --server-side --force-conflicts`, lo que delegó el cálculo de gestión de campos al API Server sin desbordar el buffer del cliente.

¿Qué haría distinto si tuviera que repetir esta práctica?

Configuraría desde el inicio un controlador de secretos sellados (Bitnami Sealed Secrets) para almacenar de forma cifrada las credenciales en el mismo repositorio de Git, evitando la necesidad de crear secretos imperativos manualmente antes de sincronizar la Application de Argo CD.

Declaración de autoría: Declaro que las capturas presentadas fueron tomadas en mi propio equipo y que las respuestas son de mi autoría. Entiendo que presentar evidencias de otra persona anula la Parte 1 de esta práctica.

Flores Mosquera Saul Osvaldo	**21 de Octubre de 2026**
Firma del estudiante	Fecha